

CNN ① 이미지에서 패턴 찾기

합성곱의 출력 크기와 파라미터 수를 계산한다.
FashionMNIST 분류기를 구현하고 학습을 진단한다.
특징 맵과 오분류를 근거로 모델의 한계를 설명한다.

IMAGE
→ **CLASS**

00-02분 · 이번 주에 만들 결과물 확인

90분 수업 지도

- 01 00-05 도입 · 05-20 개념 · 20-35 수식과 손계산
- 02 35-50 구현 설계 · 50-75 실습 · 75-85 분석
- 03 85-90 회고 · 부록과 자율 과제는 수업 이후

90 MIN / 개념 45 + 실습 25 + 분석·회고 20

02-05분 · 실습 노트북 열기

이미지 분류를 함수로 쓰기

이미지

$B \times 1 \times 28 \times 28$

특징 추출

C개 점수

argmax

- 01 입력은 픽셀 텐서, 출력은 클래스별 점수다.
- 02 학습은 이미지와 정답의 대응을 반복해서 조정한다.
- 03 추론에서는 가장 큰 점수의 클래스를 선택한다.

05-20분

MLP에서 CNN으로

- 01 Flatten은 정보를 삭제하지 않지만 2차원 이웃 구조를 직접 활용하지 않는다.
- 02 전결합층은 위치마다 별도 가중치를 가진다.
- 03 CNN은 지역 연결과 가중치 공유를 구조에 넣는다.

784 → 128: 100,480개 / Conv 1→16, 3×3: 160개

05-20분

필터 하나가 하는 일

지역 패치

3×3

곱하고 합하기

스칼라 반응

전체 특징 맵

- 01 작은 창과 필터를 원소별로 곱한 뒤 합한다.
- 02 같은 필터를 이미지 여러 위치에 적용한다.
- 03 출력 특징 맵은 특정 패턴에 대한 반응 지도다.

05-20분

채널은 무엇인가

- 01 RGB는 입력 채널이 3개다.
- 02 출력 채널 수는 서로 다른 필터 수다.
- 03 각 출력 채널은 모든 입력 채널의 반응을 합한다.

$W: [C_{out}, C_{in}, K_h, K_w]$

05-20분

이동에 대한 올바른 직관

- 01 같은 패턴이 다른 위치에 나타나도 같은 필터가 반응한다.
- 02 stride 1 합성곱은 경계 등을 제외하면 이동 등변성을 갖는다.
- 03 풀링·경계·stride 때문에 완전한 이동 불변성은 보장되지 않는다.

등변성: 입력 이동 → 특징 맵도 이동

05-20분

비선형성과 다운샘플링

Conv

ReLU

MaxPool

Conv

ReLU

- 01 ReLU는 음수 반응을 0으로 바꾼다.
- 02 MaxPool은 작은 영역의 최대값을 남긴다.
- 03 공간 해상도를 줄이면 계산량과 세부 정보가 함께 줄어든다.

05-20분

데이터 분할부터 시작

- 01 train은 가중치 학습에 사용한다.
- 02 validation은 구조와 학습 설정 선택에 사용한다.
- 03 test는 선택을 끝낸 모델의 마지막 평가에만 사용한다.

train $\neq$ validation $\neq$ test

05-20분

합성곱 한 칸 손계산

- 01 패치 $\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ 와 필터 $\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$ 을 사용한다.
- 02 원소별 곱은 1, 0, 0, -4다.
- 03 합은 -3이며 bias가 1이면 출력은 -2다.

$$y = \sum \text{patch} \times \text{weight} + \text{bias} = -2$$

20-35분

출력 크기 공식

- 01 입력 H, 커널 K, 패딩 P, stride S를 확인한다.
- 02 dilation D까지 포함하면 유효 커널은 $D(K-1)+1$ 이다.
- 03 가로와 세로에 공식을 각각 적용한다.

$$H_{out} = \text{floor}((H + 2P - D(K-1) - 1) / S + 1)$$

20-35분

형상 계산 연습

- 01 28×28 , $K=3$, $P=1$, $S=1$ 이면 28×28 이다.
- 02 2×2 MaxPool, $S=2$ 를 거치면 14×14 다.
- 03 두 블록 후 $32 \times 7 \times 7$ 을 펼치면 1,568차원이다.

$$[B, 1, 28, 28] \rightarrow [B, 16, 14, 14] \rightarrow [B, 32, 7, 7]$$

20-35분

파라미터 수를 계산하기

- 01 Conv의 bias는 출력 채널마다 하나다.
- 02 공간 크기가 커져도 같은 필터를 공유한다.
- 03 풀링과 ReLU에는 학습 파라미터가 없다.

$$\text{Conv params} = C_{\text{out}} \times (C_{\text{in}} \times K_h \times K_w + 1)$$

20-35분

수용 영역을 넓히기

- 01 3×3 합성곱 두 층은 stride 1에서 5×5 영역을 본다.
- 02 중간 ReLU가 추가적인 비선형성을 만든다.
- 03 stride와 pooling이 있으면 누적 간격도 추적한다.

$$r_{\text{new}} = r_{\text{old}} + (K-1) \times \text{jump_old}$$

20-35분

분류 손실과 확률

- 01 CrossEntropyLoss는 logits와 정수 라벨을 받는다.
- 02 내부적으로 log-softmax와 정답 항의 음의 로그를 계산한다.
- 03 학습 직전 softmax를 따로 적용하지 않는다.

$$L = -\log \text{softmax}(\text{logits})[y]$$

20-35분

오늘의 SmallCNN

$28 \times 28 \times 1$

$14 \times 14 \times 16$

$7 \times 7 \times 32$

64

10 logits

- 01 첫 블록은 $1 \rightarrow 16$ 채널, 둘째 블록은 $16 \rightarrow 32$ 다.
- 02 각 블록은 3×3 Conv, ReLU, 2×2 Pool로 구성한다.
- 03 분류기는 $1,568 \rightarrow 64 \rightarrow 10$ 으로 연결한다.

35-50분

학습 루프의 다섯 단계

이전 기울기를 비우고 forward를 계산한다.

손실을 계산하고 backward로 기울기를 만든다.

optimizer.step으로 파라미터를 갱신한다.

```
optimizer.zero_grad()
logits = model(x)
loss = F.cross_entropy(logits, y)
loss.backward()
optimizer.step()
```

35-50분

train과 eval을 구분하기

- 01 `model.train()`은 학습 모드로 설정한다.
- 02 `model.eval()`은 Dropout·BatchNorm의 동작을 바꾼다.
- 03 `no_grad`는 기울기 기록을 끄는 별도 기능이다.

`eval() ≠ no_grad()`

35-50분

평균 손실을 제대로 집계

- 01 배치 평균에 배치 크기를 곱해 누적한다.
- 02 마지막 작은 배치까지 실제 표본 수로 나눈다.
- 03 정확도는 맞힌 개수 / 전체 개수다.

$$\text{epoch_loss} = \Sigma(\text{batch_loss} \times \text{batch_size}) / N$$

35-50분

디버깅의 순서

- 01 입력의 shape·dtype·범위를 먼저 출력한다.
- 02 한 배치 forward, loss, backward가 되는지 확인한다.
- 03 소량 데이터에 과적합할 수 있는지 검사한다.

입력 → 한 배치 → 소량 학습 → 전체 학습

35-50분

LAB 1 · 데이터와 연산 확인

- 01 practice §1–3: 환경, 분할, 이미지 격자를 실행한다.
- 02 손계산 결과와 conv2d가 같은지 확인한다.
- 03 레이어별 shape와 파라미터 수를 출력한다.

50–57분 / 산출물: 이미지 격자 + shape 표

50–75분 · practice.ipynb와 함께 진행

LAB 2 · 기준 모델 학습

- 01 practice §4–5: MLP와 CNN을 같은 분할에서 학습한다.
- 02 각 epoch의 train/validation 손실을 기록한다.
- 03 validation 손실이 가장 낮은 가중치를 보관한다.

57–68분 / 산출물: 학습 곡선 + 비교 표

50–75분 · practice.ipynb와 함께 진행

LAB 3 · 틀린 예측 들여다보기

- 01 practice §6–7: confusion matrix와 오분류를 시각화한다.
- 02 첫 Conv의 활성화를 원본과 비교한다.
- 03 모델 선택을 끝낸 뒤 test를 한 번 평가한다.

68–75분 / 산출물: 오분류 8장 + 해석 2문장

50–75분 · practice.ipynb와 함께 진행

학습 곡선으로 원인 좁히기

- 01 train·validation 손실이 모두 높으면 학습 부족을 의심한다.
- 02 train만 계속 좋아지면 과적합 가능성을 본다.
- 03 validation 진동은 작은 표본·큰 lr 등도 원인이 될 수 있다.

관찰 → 가설 → 한 변수 변경 → 재실험

75-85분

정확도 하나로 충분할까

- 01 클래스별 표본 수와 recall을 함께 본다.
- 02 confusion matrix는 행이 정답, 열이 예측이다.
- 03 정답률이 같아도 실패하는 클래스가 다를 수 있다.

$$\text{recall}_c = \text{TP}_c / \text{실제 클래스 } c \text{의 개수}$$

75-85분

실험 기록을 남기는 최소 단위

- 01 데이터 분할·seed·모델·lr·epoch를 남긴다.
- 02 최고 validation과 마지막 epoch를 구분한다.
- 03 결과 차이가 작으면 여러 seed로 다시 비교한다.

한 줄 결론 + 결과 표 + 재현 설정

75-85분

출구 퀴즈 · 근거까지 말하기

- 01 Q1. 32×32 , $K=3$, $P=0$, $S=2$ 의 출력 크기는?
- 02 Q2. Conv $3 \rightarrow 8$, 3×3 , $\text{bias}=\text{True}$ 의 파라미터 수는?
- 03 Q3. CrossEntropyLoss 전에 softmax를 넣어야 할까?

1분 개인 답 → 1분 짝 설명

85-90분

다음 주로 가져갈 질문

- 01 모델을 더 깊게 하면 언제 도움이 될까?
- 02 증강이 validation 정확도를 반드시 높일까?
- 03 같은 예산으로 개선 여부를 어떻게 확인할까?

자율 과제: 커널·채널 비교 + 오류 분석

85-90분

부록 · padding의 경계 효과

- 01 zero padding은 바깥을 0으로 가정한다.
- 02 reflection padding은 주변을 반사한다.
- 03 경계의 가정은 출력 특징에 영향을 준다.

같은 크기 유지 $\neq$ 경계 정보 보존

수업 외 심화

부록 · 1×1 합성곱

- 01 공간 이웃을 섞지 않고 채널을 조합한다.
- 02 각 위치에 같은 선형 변환을 적용한다.
- 03 채널을 줄이면 뒤의 3×3 계산량을 줄일 수 있다.

$$[B, C_{in}, H, W] \rightarrow [B, C_{out}, H, W]$$

수업 외 심화

부록 · 계산량 추산

- 01 출력 한 원소는 $C_{in} \times K^2$ 개의 곱셈을 요구한다.
- 02 전체 출력 원소는 $B \times C_{out} \times H_{out} \times W_{out}$ 이다.
- 03 파라미터 수가 작아도 큰 해상도에서는 계산량이 커진다.

$$\text{MACs} \approx B H_{out} W_{out} C_{out} C_{in} K^2$$

수업 외 심화

부록 · Global Average Pooling

- 01 채널별 공간 평균을 구해 하나의 값으로 만든다.
- 02 입력 해상도 변화에 대응하기 쉬워진다.
- 03 정밀한 위치 정보는 평균 과정에서 줄어든다.

$[B,C,H,W] \rightarrow [B,C,1,1]$

수업 외 심화

부록 · 특징 맵 읽기의 한계

- 01 한 채널의 활성화는 표현의 일부다.
- 02 밝은 위치가 예측의 유일한 원인은 아니다.
- 03 모델·입력 변형에 따른 출력 변화도 함께 관찰한다.

활성화 시각화는 진단 도구다

수업 외 심화

부록 · tiny overfit 점검

- 01 32개 표본만 반복해서 학습한다.
- 02 규제를 잠시 줄여 학습 가능한지 본다.
- 03 거의 외우지 못하면 파이프라인 오류를 먼저 찾는다.

디버깅용 과적합 $\neq$ 일반화 평가

수업 외 심화

부록 · 자율 과제 설계

- 01 기본: 출력 크기와 파라미터 함수 구현.
- 02 실험: 채널 폭을 바꾸고 같은 예산으로 비교.
- 03 도전: 이동 후 예측 변화와 오분류 사례를 분석.

근거 없는 최고 정확도 경쟁은 피한다

수업 외 심화

시각 보충 · 패치와 필터의 곱

입력 패치

1	2
3	4

×

공유 필터

1	0
0	-1

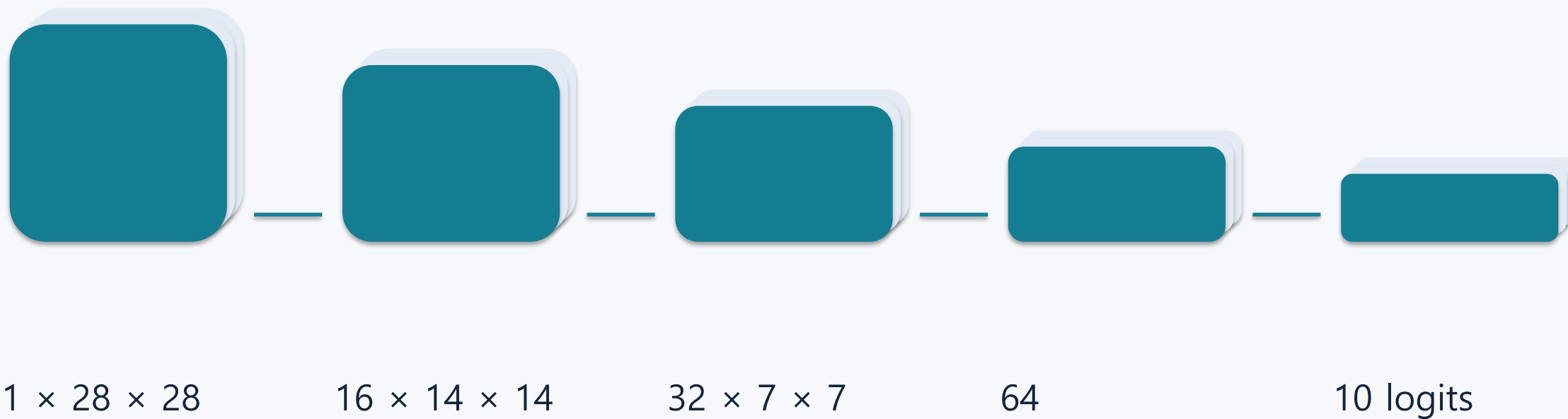
=

$$1 \times 1 + 2 \times 0 + 3 \times 0 + 4 \times (-1) = -3$$

bias=1을 더하면 -2 / 같은 필터를 다음 패치로 이동한다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · 해상도와 채널의 흐름



B는 모든 단계에서 유지된다. 채널 수와 공간 해상도를 서로 바꾸어 읽지 않는다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · 혼동 행렬 읽기

설명용 3-class confusion matrix

	예측 A	B	C
정답 A	8	1	1
B	2	6	2
C	0	3	7

$$B \text{ recall} = 6 / (2+6+2) = 0.60$$

$$B \text{ precision} = 6 / (1+6+3) = 0.60$$

대각선: 정답

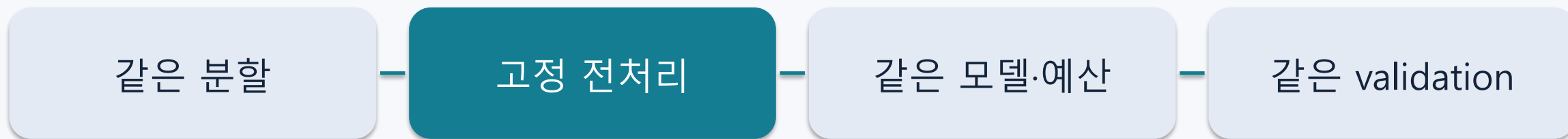
행 합: 실제 클래스 표본 수

열 합: 그 클래스로 예측한 수

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

시각 보충 · 공정한 실험의 두 경로

A · 기준



B · 변경



기법의 이름보다 무엇을 고정했고 무엇을 바꿨는지 먼저 기록한다.

수업 외 복습 · 해당 개념 설명 시 함께 활용 가능

읽을거리와 출처

- 01 PyTorch · 이미지 분류 튜토리얼
- 02 Conv2d · 공식 API
- 03 Deep Residual Learning · He et al.

원문 링크는 발표자 노트와 주차별 README에 수록

생각하기 → 손으로 계산하기 → 코드로 검증하기